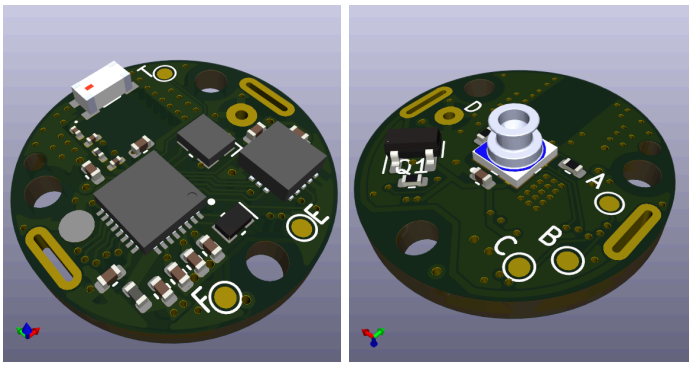
Project Ember Spring 2026 Progress Report

Illini Solar Car



February 2026 - May 2026

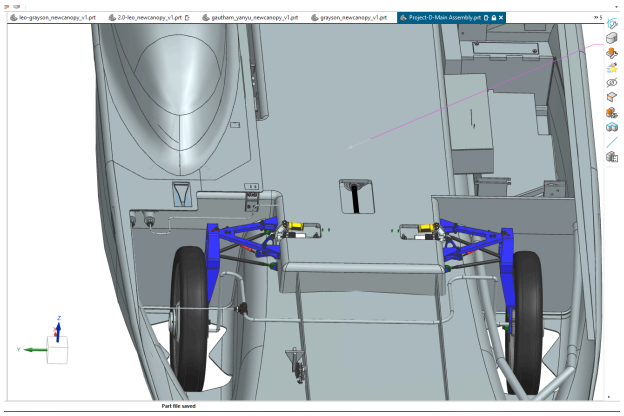
Author

Eddie Tang

Project Overview

For this project report, we will detail our goals, current progress, and plans for future designs and hardware, along with the design rationale and decisions we made that have brought us thus far.

Project EMBER (Embedded Micro-power Bluetooth Energy Research) is a joint Illini Solar Car x AbbVie research project to design an ultra-low-power TPMS prototype as a vehicle for studying the implementation and usage of low-power protocols and operating modes that have not been fully explored. This project proposes two deliverables: a functional TPMS prototype (wheel-mounted transmitter + receiver) and a technical research report with lessons learned, engineering decisions, and recommendations transferable to AbbVie's medical device platform.



CAD Preview of Project D, with location of wheels.

The main reason for choosing a TPMS system to study low-power design is that it best aligns with both AbbVie's requirements for what to research and what Solar Car would benefit from (in this case, real-time pressure data for vehicle dynamics). Typically, aftermarket TPMS sensors have up to four sensor nodes that transmit to one central receiver mounted in the car; however, this poses a problem, since the chassis itself is made from carbon fiber, which essentially acts as an RF shield, significantly reducing the amount of data we can receive if we place the receiver node near the typical telemetry board that the team uses. This provides us with a motivation, along with the pre-existing one, to create our own TPMS sensors and, more importantly, receiver boards.

Consequently, the subsequent sections of this report will be dedicated to the design and development of the sensor node Printed Circuit Board (PCB).

Sensorboard (a better name is yet to come)

This board is our transmitter/TPMS sensor, which we hope to mount on a tire if we can get it working for the team. This sensor board will periodically transfer data over BLE to a dedicated receiver station that is mounted close to it in the car. Moreover, it will operate in a highly asymmetric duty cycle, with long idle periods while the car is parked, punctuated by *long (up to 12 hours)* active periods while driving.

For this PCB, we selected to use a discrete TPMS design instead of using a SOC TPMS sensor. Discrete TPMS designs are those that involve using more than one IC to perform the required functionalities of a TPMS (measure pressure, auto wake, and send data). SOC TPMS sensors have everything available in one IC, reducing space and allowing for more standardization. The main advantage of a discrete design is that it offers more customization and research potential, which is what this project's primary deliverable presents, instead of a TPMS that is industry standard.

The table below provides a brief overview of the comparison.

Feature	SoC (e.g., NTM88[1])	Discrete (e.g., MCU + Pressure sensor)
Research Value	Low (Black box)	Very High (Customizable system, measurable outcomes, and characterizable research points)
Development Time	Faster (Integrated)	Slower (Requires custom drivers)
Power Potential	Slightly better (Optimized die)	Good (Depends on our engineering)

In addition to the choice of making this PCB discrete, we will be focusing on making an external TPMS sensor instead of an internal TPMS sensor. The main reason between the two is where it measures the pressure from, outside and inside the wheel, respectively. Having an external TPMS sensor means easier mountability, faster switch times, better troubleshooting, and

[1] https://www.nxp.com/products/NTM88

less worry of a counterweight on the other side of the tire if we were to install an internal TPMS sensor within the tire.

This board is the main research vehicle. It will be the main system in which we investigate and demonstrate sub-microamp standby current at the system level without sacrificing measurement reliability or BLE reachability (transmission distance of <1m).

The table below will detail key constraints for this PCB.

Constraint	Value
Power	CR1632 lithium coin cell, 120 mAh, 3V nominal
Pressure Range	≤ 85 psi
Temperatures	$15°C \leq T_{operation}\ °C \leq 40°C$
Form factor	Must fit inside gutted commercial external TPMS casing
Standby current target	≤ 100 µA total system at rest
Communication	BLE

Receiver board (yes, this one will also have a better name)

This board will be the receiver board that goes with every sensor node, and will hopefully be mounted in an RF-accessible opening in the car that will allow it to connect to the rest of the car via +24V and CAN. For the implementation aspect of this project, this board is the focus, as it deals with a direct integration of a new PCB with the car system.

The table below will detail key constraints for this PCB.

Constraint	Value
Power	+24V
Communication	BLE, CAN

Power Requirements

Most small coin cells used in embedded sensing are lithium-manganese dioxide ($Li\text{-}MnO_2$) types with a nominal voltage of 3 V and very low self-discharge, making them suitable for multi-year, low-current applications. These voltages are also suitable for many

sensors, as we have found that 3V is the typical supply voltage rated for many components we have come across.

Returning to coin cells, the IEC naming convention encodes chemistry, diameter, and height: for example, CR1632 denotes a round lithium cell (CR) with 16 mm diameter (16) and 3.2 mm height (32).[2] For a fixed chemistry, capacity scales primarily with volume, so thicker cells with the same diameter generally provide more mAh at the cost of mechanical thickness.[3]

CR1632 Justification

The table below compiles average capacities from different manufacturers along with some notes.[4][5] Notice that standard CR sizes jump from CR1632 up to CR20xx, with no other thickness in between.

Cell type	Diameter (mm)	Height (mm)	Typical capacity (mAh)	Notes
CR1616	16	1.6	~45–50	Very thin, used in slim remotes and watches.
CR1620	16	2.0	~70–80	Moderate thickness, ~75 mAh typical.
CR1632	16	3.2	~120–140	High-capacity 16 mm cell; many guides quote ~120–130 mAh, some ~140 mAh.
CR2016	20	1.6	~80–90	Larger diameter, still thin; ~90 mAh typical.
CR2032	20	3.2	~210–240	Very common; ~220 mAh typical, but 20 mm diameter.

So far, teardowns of low-cost aftermarket external TPMS kits have shown that many of them use CR1632 cells. Also, among mechanically compatible 16 mm cells, CR1632 simply gives us the largest energy budget without changing PCB footprint at all. This convergence between both the size and energy suggests that a ~130 mAh CR1632 cell provides a proven

[2] https://en.wikipedia.org/wiki/List_of_battery_sizes#Lithium_cells
[3] https://www.lisleapex.com/blog-cr1620-vs-cr1632-what-are-differences-and-how-to-choose
[4] https://en.wikipedia.org/wiki/List_of_battery_sizes
[5] https://batteryspecialists.com.au/blogs/news/cr1632-batteries-what-you-need-to-know,-before-buying-this-coin-cell

trade-off between mechanical size and multi-year operating life in real driving conditions, which matches the goals of this project.

It is important to note the characteristics of the CR1632 that we bought, as it starts off at 3.3V, and with more drain, settles at a nominal 3.0V, until it finally dips to ~2.2V.

For BOM 1, we ordered the cheapest CR1632 batteries from LiCB via Amazon, which come with the standard dimensions with a capacity of 120mAh.

Operational Cycle Overview

The SensorBoard operates in a motion-gated duty cycle driven by the fundamental asymmetry of its use case: a car spends the vast majority of its time parked, meaning any current drawn during inactivity directly dominates battery life. The optimal strategy, therefore, is to minimize the time the system spends outside of its lowest-power state, and not through a fixed wake interval, but by delegating the wake decision entirely to hardware. When the car is parked, the system sits in its deepest sleep state indefinitely, drawing sub-microamp current, and only exits that state when motion is physically detected.

At a high level, the cycle has three phases. (1) When the car is stationary, the system is fully asleep and waiting for a motion event. (2) When motion is detected, the system wakes, begins recording and transmitting pressure data over BLE at a periodic interval, and continues doing so as long as the car remains in motion. (3) When the car stops and sustained inactivity is confirmed, the system takes a final measurement, transmits it, and returns to its deepest sleep state — where it remains until the next motion event. The details of how each transition is implemented in hardware and firmware are covered in the Hardware Architecture and Firmware Power Management sections, respectively.

In simpler terms, the car:

1. Records & transmits pressure data via BLE
2. Detects inactivity, goes back to sleep
3. Detects activity, wakes up, repeats (1)

IC & Sensor Justification

To implement our 3-phase cycle, we will employ an accelerometer for motion detection, a high-pressure barometer for tire pressure data, and an MCU that supports integrated BLE communication.

The table below provides a brief overview of our chosen components.

Functionality	Chosen component	Link
MCU	EFR32BG22E224F512IM32	Click me
Barometer	MS583730BA01-50	Click me
Accelerometer	ADXL362	Click me

Applicable Terms

We found that vendors like to advertise their "low power modes" differently. However, they typically provide the same functionalities and key metrics, such as current consumption, output data rate, and noise. When looking for a sensor that would work, we would recommend looking at quantitative benchmarks as listed above instead of being swayed by marketing terms that different manufacturers use to describe essentially the same power-conservation outcome.

Term	Definition & Notes
ODR f, Hz	Output Data Rate: the frequency at which the accelerometer updates its output data. In general, increasing ODR increases bandwidth and responsiveness, but it also tends to increase current consumption
OSR N	Oversampling Rate: the ratio between the internal sampling rate and the minimum sampling rate required by Nyquist[6]. Higher OSR can improve noise performance and effective resolution, but it usually increases power and processing overhead and time.
Term	Definition & Notes (Accelerometer specific)
Noise	Random variation or jitter in the output that limits how cleanly small motions can be measured. Lower noise improves the ability to detect subtle acceleration changes, but achieving lower noise often requires more current or lower bandwidth

[6] https://en.wikipedia.org/wiki/Oversampling

Bandwidth	The frequency range over which the accelerometer can accurately respond to motion. Higher bandwidth allows faster changes in acceleration to be captured, but it usually increases noise and power consumption; in many accelerometers, bandwidth is tied to ODR and the antialiasing filter setting
Measurement mode (Active)	The normal active sensing state in which the accelerometer continuously samples motion and outputs data. This mode is used when full bandwidth, regular data updates, and continuous monitoring are required.[7]
Low power mode (Active, but less accurate)	A broad vendor-specific term for a reduced-power operating state. Depending on the device, this may mean lower ODR, reduced bandwidth, duty-cycled sensing, or simplified internal processing, often at the cost of responsiveness or data quality.[8]
Standby mode (Light sleep)	A term multiple manufacturers like to use. A very low-power idle state in which active measurement is suspended, but device configuration is typically retained. Standby mode allows for a quick return to active mode.[9]
Power down mode (Medium sleep)	A term that ST uses. A near-off state in which most internal sensing and processing blocks are disabled to minimize current draw. Register content is preserved, but there are no measurements performed.[10]
Hibernate mode (Dead sensor)	A term used to describe NXP products. Components in this mode have a deeper sleep than in the standby mode. However, an external pin is required to trigger a wake-up.
Wake-up mode (Auto sleep & active)	A term that ADI (Analog Devices Inc) uses. A low-power monitoring state in which the accelerometer samples infrequently or uses simplified detection to look for motion. It automatically wakes up from motion detection and transitions into the desired active mode. This term is similar to what NXP calls “Active sleep mode”[11].

MCU - EFR32BG22 Family

The primary constraints for MCU selection were ultra-low sleep current, an integrated BLE radio, a small form factor, and alignment with Silicon Labs' ecosystem, which is the same platform AbbVie uses in their medical devices, as well as what they want to investigate for the future, making findings and firmware patterns directly transferable.

[7] Datasheet Example
[8] Datasheet Example
[9] Datasheet Example
[10] Datasheet, https://github.com/gschorcht/lis3dh-esp-idf/blob/master/README.md
[11] Datasheet

The leading alternative considered was the Nordic nRF52840. While it is a capable and popular BLE MCU with significantly more RAM and a higher clock ceiling, it is optimized for computationally heavier BLE workloads. For this project, raw performance is not a bottleneck, since the SensorBoard spends the vast majority of its time in deep sleep, and the nRF52840's larger die and higher active current are liabilities, not assets. Silicon Labs' architecture places a stronger emphasis on sleep-state efficiency and on-chip power management, which better matches the asymmetric duty cycle described in the Operational Cycle Overview.

We selected the **EFR32BG22E224F512IM32** (QFN32) specifically. A notable secondary reason for this package choice is that it is only one of the two QFN32 variants (EFR32BG22E224F512IM32, EFR32BG22C224F512IM32) that carry AEC-Q100 automotive qualification, meaning it has been stress-tested for the thermal cycling, vibration, and humidity conditions that a wheel-mounted sensor will experience.

Key specifications are summarized below:

Parameter	Value
Core	ARM Cortex-M33
Max Clock	76.8 MHz
Flash / RAM	512 kB / 32 kB
Package	QFN32
TX Current @ 0 dBm	4.1 mA
RX Current	3.6 mA
RX Sensitivity	−98.9 dBm (1 Mbit/s GFSK)
Digital I/O	18 pins
Development Environment	SimplicityStudio

Barometer - MS5837

The primary constraints for pressure sensor selection were a measurement range covering at least 85 PSI, low active and standby current, small physical dimensions, and suitability for an externally mounted TPMS casing.

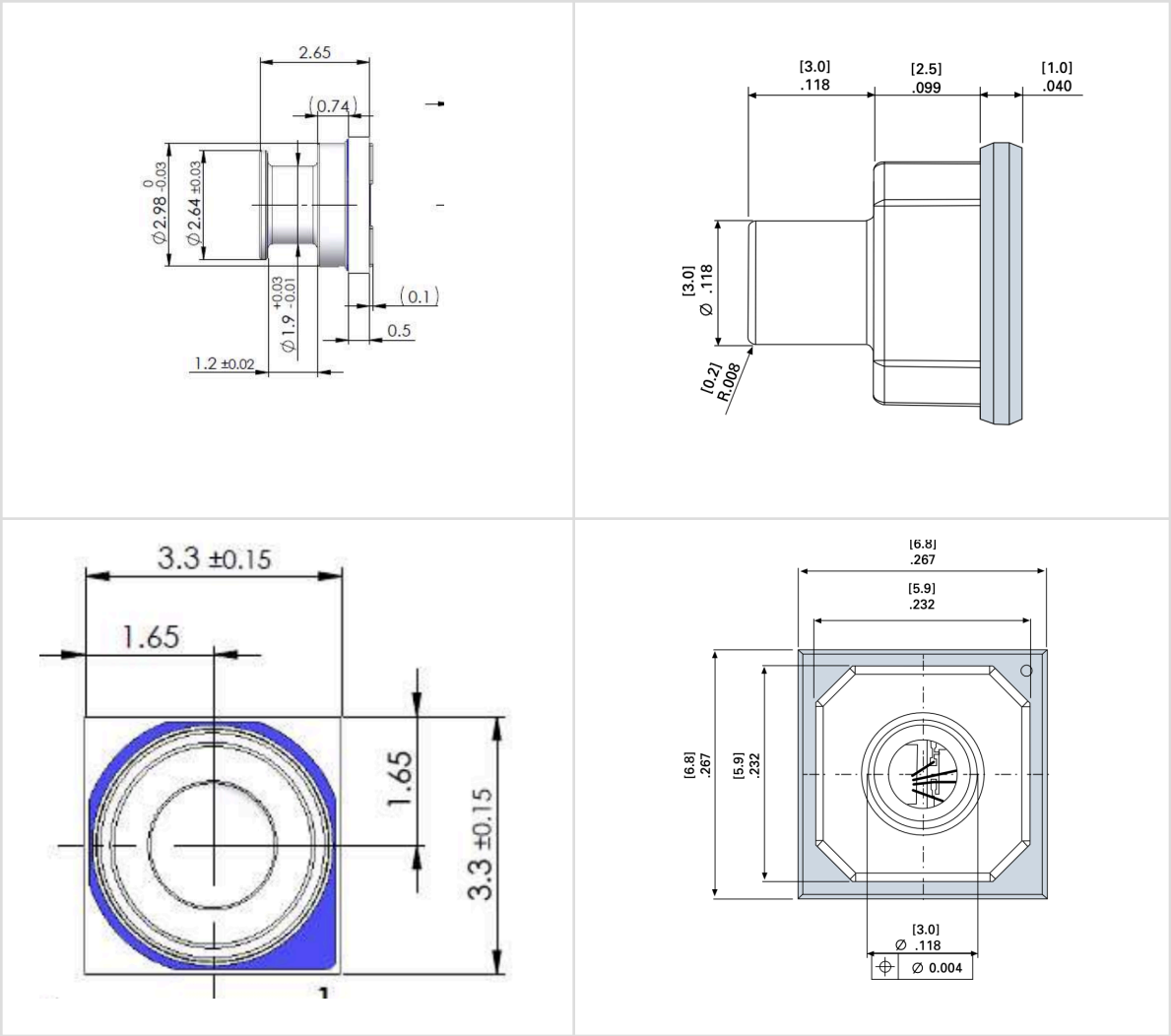
The most important constraint, and the one that eliminates nearly every commonly cited low-power pressure sensor outright, is pressure range. Car tires operate at 30-35 PSI. The majority of MEMS barometric pressure sensors found in consumer electronics are rated only to approximately 1.25 bar (~18 PSI) and are physically incapable of measuring tire pressure.

The Merit Sensor CMS technically covers the pressure range but draws ~3 mA active, three orders of magnitude above the MS5837's active-mode current, making it incompatible with a coin-cell budget. This leaves the MS5837-30BA as effectively the only viable low-power digital MEMS pressure sensor for this application. Furthermore, the CMS is much larger than the MS5837, again making it less favorable for use.

Sensor	Max Pressure	Active Current @ 1Hz
MS5837-30BA[12]	30 bar (435 PSI)	0.63 µA (OSR 256)
Bosch BMP390[13]	1.25 bar (18 PSI)	3.2 µA
ST LPS22HH[14]	1.26 bar (18 PSI)	4 µA
Merit Sensor CMS[15]	10 bar (150 PSI)	~3 mA (active)

MS5837 | mm	CMS | inch [mm]

[12] Datasheet
[13] Datasheet, (An aftermarket TPMS uses this)
[14] Product Webpage
[15] Datasheet



It is worth noting that the MS5837-30BA is deliberately over-ranged for TPMS use: tire pressure sits at roughly 3-4 bar, which is only ~10% of the sensor's 30-bar full-scale. This has two practical consequences. First, absolute accuracy at the operating pressure range (±50-100 mbar, or approximately ±0.7-1.5 PSI) is the best the sensor offers; its accuracy degrades at higher pressures, but we never operate there.[16]

The known weaknesses of this sensor that will impact firmware implementation are worth documenting: the MS5837-30BA is I²C-only with a fixed address of 0x76 and no hardware interrupt, FIFO, or data-ready pin. Every measurement requires a software-timed sequence of two separate ADC conversion commands (D1 for pressure, D2 for temperature), each followed by a polling wait of up to 18 ms at maximum OSR. Additionally, the datasheet

[16] Datasheet

documents an ~8 mbar calibration offset that can occur post-reflow soldering, with a 2-7 day recovery window, which is something that must be accounted for during board bring-up and calibration.

Accelerometer - ADXL362

The primary constraints for accelerometer selection were ultra-low wake-up current, small dimensions, and, most importantly, the need for the accelerometer to detect motion and drive a wake/sleep transition of the system. It does **not** need to log high-fidelity[17] acceleration data.

Sensor Options	Power draw
ADXL362[18]/ADXL367[19] (ADI)	1.8µA/0.89µA (100Hz ODR) | 0.27µA/0.18µA (ADI-proprietary wake-up mode/6Hz DR)
ST LIS3DH[20] (ST)	6µA (50 Hz ODR low-power mode) 0.5µA (power-down mode)
ST IIS2DLPC[21] (ST)	50 nA in power-down mode below 1 µA in active low-power mode 120 µA in high-performance mode
FXLS8974CF[22] (NXP, now part of ST)	≤ 1 µA (6.25Hz ODR) 650 nA in standby mode 50 nA in hibernate mode

The table above highlights many of the top choices for an accelerometer to be found in a TPMS in terms of typical power draw. The constraints immediately rule out many of the options, for instance, the ST IIS2DLPC optimizes both low-noise precision and low power, creating a dynamic where the additional power-draw, despite being very low already, will be wasted since the main functionality of the accelerometer will be waking up the system from sleep.

As a digression, despite not using the ST IIS2DLP for our PCB, we found that it has been recommended by ST as a commercial option for a robust, low-power accelerometer for vehicle

[17] High resolution and low noise.
[18] Datasheet
[19] Datasheet
[20] Datasheet
[21] Datasheet
[22] Datasheet

applications where data is important. Moreover, while doing research, we found a practical and useful guide released by Analog Devices that describes the process of choosing the best MEMS sensor for wireless application-specific requirements, which may be more applicable in commercial R&D.

On the other hand, the BMA250 and ST LIS3DH are commonly cited as low-power options, but both use duty-cycling to achieve low current figures: their analog front-ends gate off between samples, meaning signals between polling intervals are simply missed. ADI's own competitive comparison document overstates this as a universal flaw[23]; however, for a TPMS specifically, where all that matters is detecting sustained wheel rotation at roughly 6 Hz, duty-cycling is a real limitation that can produce false negatives on short motion events. Parts that sample continuously at all ODRs, like the ADXL362, are architecturally better suited to this use case.

This elimination brings out the ADXL362 and NXP FXLS8974CF accelerometers. At a 6 Hz comparison point, the ADXL362's dedicated wake-up mode draws only 270 nA, which is approximately 3.7x lower than the FXLS8974CF's minimum active-sampling current of 650-1000 nA. Though for credits, the FXLS8974CF does offer a 15 nA hibernate mode, but that requires an external pin trigger from the MCU rather than autonomous motion detection, which defeats the purpose of hardware-delegated wakeup in this design. In any other low-power operational setting, we would suggest re-evaluating the FXLS8974CF and ADXL362 for company-specific benchmarks. For instance, neither of these sensors offers a large acceleration range. However, the acceleration range suits the PCB requirements, and low power consumption is necessary, so it is worthwhile looking at FIFO buffer length, since it can help dictate the polling interval to that sensor, and reduce power consumption significantly.

The table below summarizes key quantitative comparisons between the two sensors.

Parameter	ADXL362	ST FXLS8974CF
Supply Voltage	1.6 V – 3.5 V	1.71 V – 3.6 V
Measurement Mode Current (100 Hz ODR)	1.8 µA (Normal), 3.3 µA (Low Noise)	5.3 µA (High Performance Mode)

[23] https://ez.analog.com/mems/w/documents/4198/adxl362-current-usage-versus-competitors

Wake-Up Mode Current	270 nA @ ~6 Hz	≤ 1 µA @ ODRs up to 6.25 Hz (Low-Power Mode)
Standby Current	10 nA	600 nA
Resolution	12-bit, 1 mg/LSB (±2g range)	12-bit, 0.98 mg/LSB (±2g range)
Measurement Ranges	±2g, ±4g, ±8g	±2g, ±4g, ±8g, ±16g
FIFO Depth	512 samples	32 samples (144 bytes)
Interface	SPI	I2C (up to 1 MHz), SPI (up to 4 MHz)
Package	3.0 × 3.25 × 1.06 mm LGA	2.0 × 2.0 × 0.95 mm DFN-10
Shock Survival	5000g (powered and unpowered)	2,000g (0.5 ms), 10,000g (0.1 ms)

One important note is that we also listed in the table the ADXL367 along with the ADXL362 (our initial rounds of research did not yield us the ADXL367, and it was added during the writing of this report). Specifically, the ADXL367 is a successor to the AXL362 and was released by ADI in 2024. On basically all aspects from the datasheets, the ADXL367 is better for our application.

Parameter	ADXL362	ADXL367
Current @ 100 Hz ODR | Normal operation	1.8 µA	0.89 µA
Wake-up mode	270 nA	181 nA
Standby current	**10 nA**	**40 nA**
Resolution	12-bit, 1 mg/LSB	14-bit, 0.25 mg/LSB
Min. supply voltage	1.6 V	1.1 V
Package	3.0 × 3.25 × 1.06 mm	2.2 × 2.3 × 0.87 mm
Interface	SPI only	SPI + I²C

Stock at time of writing	Available	Zero stock

Despite these strong reasons to pivot to the ADXL367, staying with the ADXL362 is more of an engineering argument for this project's current state at the time of writing.

First, the PCB schematic and layout are already completed for the ADXL362's LGA footprint, and are under review at the time of writing. Using a different footprint, it would require extra time and may require a heavy re-layout, pushing our schedule back further.

Second, ADI still lists the ADXL362 as "Recommended for New Designs" on their webpage, meaning (1) it would not be deprecated soon and (2) teams, such as ours, with not much experience with the ADXL sensor suite, would have an easier time working with the ADXL362 than the ADXL367.

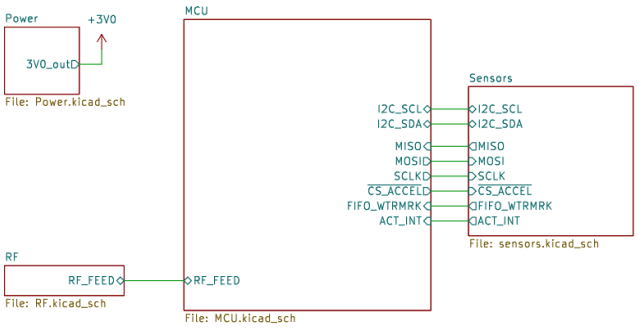
Third, as bold in the table, the ADXL367's standby current is 4x worse than the ADXL362's 10nA, which partially offsets its ~2x savings in active mode and wake-up mode.

Finally, at the time of component selection, the ADXL367 had zero stock, making it an impractical choice regardless of its spec advantages.

Therefore, the decision to use the ADXL362 is defensible, and if AbbVie is considering working with nanopower accelerometers for continuous, low-acceleration sampling, then the ADXL367 is a solid choice.

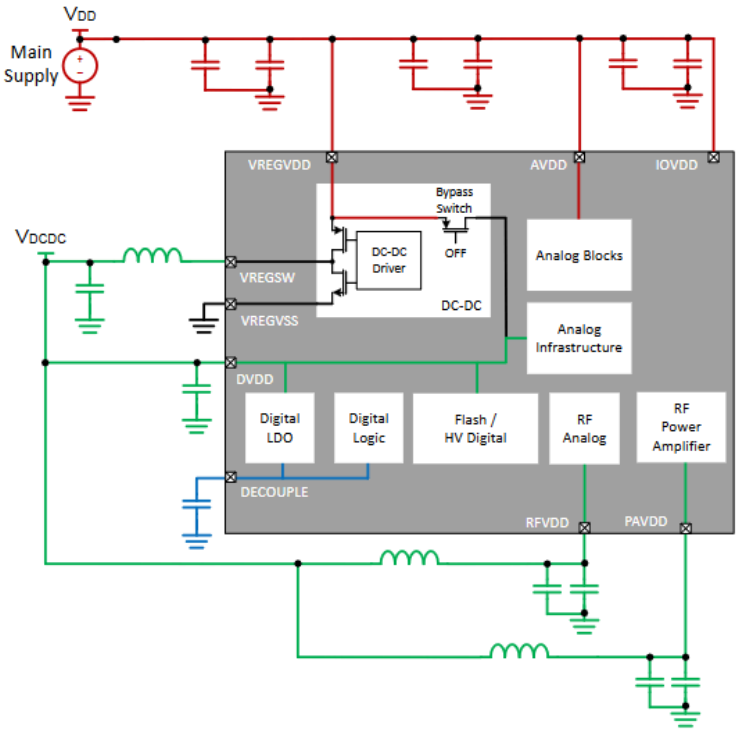
Hardware Architecture

Schematic Summary



A high-level overview of the different subsystems

Power configuration



[24]

Recommendations

Recommendation 1: Despite not being implemented in our initial design, we recommend using chips or sensors that have an enable toggle pin that can be written to, which will completely put the sensor into deep sleep. If that is not possible, we recommend verifying the current draw and seeing whether a MCU pin can provide that much current, and directly powering the sensor through the GPIO pin. This creative approach allows for power toggling without additional circuitry (compared to using a MOSFET) and directly reduces power consumption.

Other recommendations may be embedded in the text.

VDD vs VDCDC - 3V or 1.8V?

The EFR32xG22 chip family supports an internal DC-DC buck regulator that steps down a VREGVDD input voltage between 1.8 V and 3.8 V to a nominal 1.8 V output. For this design, VDD = 3.0 V and VDCDC = 1.8 V.

[24] Page 292 of EFR32xG22 Reference Manual

Regarding power pin wiring, each supply pin on the EFR32 can be connected to either VDD or VDCDC. Page 25 of the EFR32xG22 datasheet specifies that connecting all supply pins to a 3.0 V VDD with a 1.8 V VDCDC yields the lowest power consumption configuration. The schematic for the EFR32xG22 Explorer Kit[25] reflects a similar configuration, though with some pins connected to VDD rather than VDCDC, which is understandable for a development kit not optimized for power. For detailed voltage constraints on each pin, refer to Page 289 of the Reference Manual.

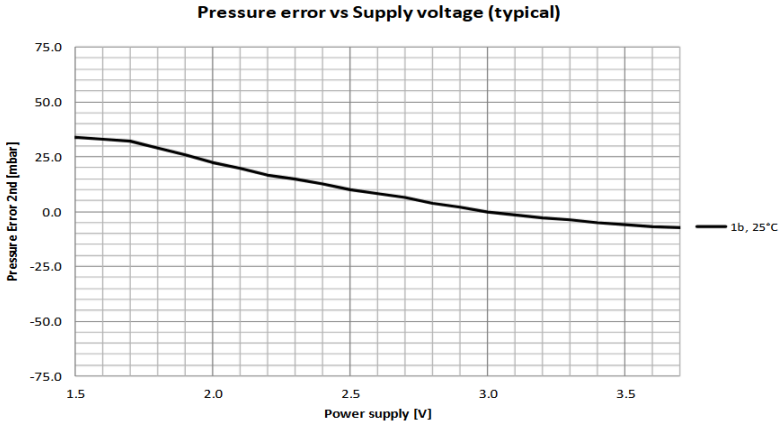
Several important notes apply to the use of the DC-DC regulator. First, if the DC-DC regulator is not used, the VREGSW pin must be left as no-connect (NC). Second, if it is used, an LC filter circuit must be connected to VREGSW, which outputs VDCDC (1.8 V). We use the values provided in the EFR32 datasheet, L = 2.2 µH and C = 4.7 µF. Third, the DC-DC regulator must be enabled in software - most likely handled by the pre-existing drivers in Simplicity Studio - which means that during power-on reset (POR), the bypass switch is automatically active, disabling the DC-DC converter and connecting the internal supply rails directly to VDD. Fourth, the Reference Manual (near Page 293) provides a detailed walkthrough for using the internal comparator to determine whether the DC-DC bypass should remain enabled, since the DC-DC converter has a minimum input voltage requirement. This will be implemented in firmware, as it allows the system to continue operating on the coin cell further into its discharge curve.

While connecting VDCDC to all supply pins is the most straightforward approach for minimizing power, it imposes constraints on other system functionalities.

For one, measuring battery voltage over time is typically accomplished using the chip's ADC with an external voltage divider. This chip, however, includes an IADC capable of measuring the AVDD source directly. If AVDD is connected to VDCDC, this measurement becomes meaningless for battery monitoring purposes, since the DC-DC regulator's function is to hold the output at a stable 1.8 V regardless of input voltage variation.

[25] There are two explorer kits, one called "EFR32xG22E Explorer Kit" and another called "BG22 Bluetooth SoC Explorer Kit," equipped with a EFR32MG22E and EFR32BG22, respectively. Silicon labs released schematics for both, but only the layout for the kit with the EFR32BG22 MCU. For the rest of this report, references to the explorer kit's layout refers to the BG22 Explorer Kit, while references to its schematic refers to both, as wiring related to the MCU on both boards are nearly identical.

Second, connecting IOVDD to 1.8 V requires all external sensors to communicate using 1.8 V logic levels. Examining the MS5837-30BA datasheet, there is approximately a 30 mbar error difference between a 1.8 V and 3.0 V supply voltage. Given that a TPMS's function is to measure pressure accurately, we would need a 3.0V input and, by extension, communicate with 3.0V logic.



Hence, in our final power configuration, we mirror an example power configuration in the reference manual, connecting IOVDD and AVDD to VDD to support logic at higher voltage levels.

In the table below, we list what each supply pin powers and our engineering decisions.

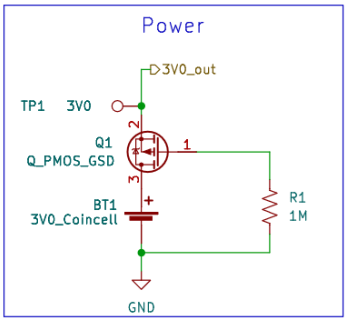
Pin	Name	Functionality & Justification
DVDD	Digital Core Supply	Main digital logic supply. CPU, SRAM, flash, and digital peripherals WITHIN the EFR32 are powered through this. Keeping at a lower voltage reduces the power draw due to switching loss.
AVDD	Analog Supply	Powers internal ADC, DAC, comparators, and analog circuits. Separated so that analog and digital noise do not interfere with each other.
IOVDD	GPIO Supply	Sets GPIO communication levels. Should be the same as the GPIO level of the sensors.
RFVDD & PAVDD	Transceiver and Power Amplifier Supply	Powers a 2.4 GHz transceiver, along with the amplifier circuit. All analog. Mirrors Explorer Kit schematic and example power configurations.

Decoupling Capacitors

Capacitor Symbol	Value	Use	Justification
C2	120pF	RFVDD	Matches Explorer Kit schematic (C12); 100 nF cap marked NM, therefore not populated.
C3	120pF	PAVDD	Matches Explorer Kit schematic (C19); 100 nF cap marked NM, therefore not populated.
C5	1uF	IOVDD & AVDD	Matches Explorer Kit values; shared cap acceptable for BLE-only design with no precision ADC use.
C7	1uF	DECOUPLE	Stability capacitor for the internal digital LDO output. Must not be connected to any supply rail. The datasheet specifies 1.0 µF minimum
C8	10uF	VREGVDD	Bulk input capacitor for the DC-DC converter input rail; handles transient load demands
C12	100nF	AXL-VS	ADXL362 datasheet-recommended bypass cap on VS supply pin. Not power hungry so 100nF is OK.
C13	100nF	MS-VDD	Standard bypass cap value for MS5837 VDD supply pin
C14	100nF	AXL-VIO	ADXL362 datasheet-recommended bypass cap on VDD_IO pin

Notice that there is no decoupling capacitor right next to DVDD in the schematic. This is because the 4.7uF capacitor is part of the filter circuit and acts as a decoupling capacitor, suppressing high-frequency noise. DVDD is connected to the same node.

Reverse Polarity Protection



Since a coin cell can easily be inserted with reversed polarity, a reverse polarity protection circuit was implemented using a reversed PMOS configuration.

The general rationale for preferring a PMOS over other protection topologies is well-established in the literature and will not be covered here. One design note worth stating is that such circuits typically include a Zener diode rated to the maximum gate-source voltage. In this design, however, the 3.0 V supply is well below the device's maximum VGSGS rating, so no zener was used.

Typically, reverse polarity protection circuits of this type commonly use a 100 kΩ pull-down resistor. A 1 MΩ value was selected instead to reduce the quiescent current draw by an additional factor of ten.

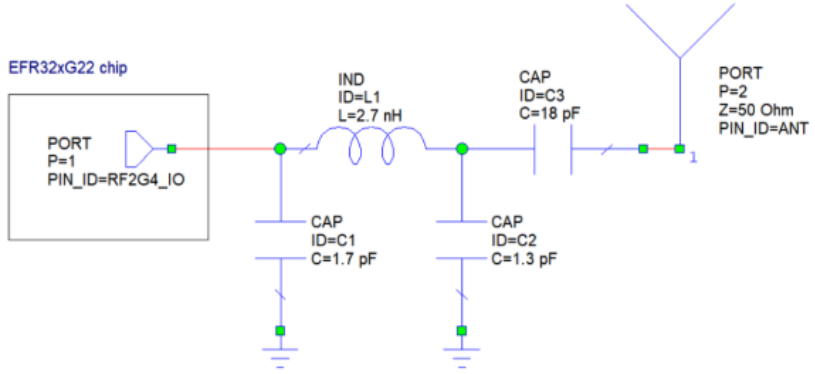
The selected PMOS is the IRLML6401TRPbF, which offers a low on-resistance of $R_{DS(ON)} = 0.085\ \Omega$ at $V_{GS} = -2.5$ V. The device comes in a SOT-23 package, which allows straightforward substitution with an alternative PMOS should a better-suited part be identified during validation.

One important caveat is that, while 85 mΩ is acceptable at low currents, the power dissipation across this device scales with the square of the current and becomes non-negligible at even a few milliamps. For AbbVie engineers adapting this design to products that do not support user battery replacement, the recommended approach is to omit this circuit entirely and enforce correct battery installation during manufacturing to eliminate the risk of user-induced reverse polarity damage.

In the future, we may bridge source and drain together via solder/wire if we find that current consumption exceeds our expectations, as detailed in the Power section.

RF

To match our MCU with the 50Ω impedance of the antenna, we consulted Silicon Labs' 2.4GHz matching guide and layout design guide for RF for the EGR32 series chips.



Pi Matching Network Schematic Following Generic Layout Concept. Page 26/43.

In this guide, they present two layout configurations: the one above is optimized for a generic layout that has a max separation of 800µm between the top and first inner layer, and another (called the existing layout concept) solely for 4-layer boards with a separation of 300µm between the top and first inner layer. For both matching networks, they can achieve power levels between 0 and +6 dBm.

One observation we found is that the RF matching network on the Explorer Kit has a different configuration compared to the recommended values. This is because they have used VNA to optimize and tune components for their specific board. Additionally, there is a discrepancy between the efr32xg22 datasheet and the matching guide. The datasheet recommends values that come from the existing layout concept, while the matching guide only recommends the existing layout concept under specific board thickness values. It is important to verify board dimensions and constraints before selecting a layout and values for the matching network.

One major discrepancy between the Explorer Kit and our board would be the lack of an antenna matching network. This is primarily due to the space constraint, and the fact that our receivers will be placed decently close to the sensor nodes during field deployment, so it is reasonable to assume that we can take a hit on our transmission signal quality.[26]

[26] Some stack exchange information

When designing for commercial BLE, we would recommend using a Pi-matching network and VNA to determine the best values for the antenna matching network.

Sensors

The I²C bus on this design carries a single follower device: the MS5837-30BA pressure sensor. The bus operates in Standard Mode (100 kHz), with a maximum rise time t_r of 1 µs per the I²C specification on page 673 of the reference manual. We connected two 10kΩ resistors on each communication line in order to reduce quiescent current.

Below, we prove that 10kΩ resistance is acceptable in our design; feel free to move on to the next section if deemed necessary.

The maximum allowable pull-up resistance is derived from the EFR32xG22 Reference Manual (also Page 637) using the following expression:

$$R_{p(max)} = \frac{t_r}{0.8473 \times C_b},$$

where t_r is the rise time, and C_b is the bus capacitance.

For an estimate of the bus capacitance in our device, it is as follows:

Source	Contribution
EFR32BG22 I/O pin	~5–10 pF
MS5837 I/O pin	~5–10 pF
PCB trace capacitance	~10–30 pF (at 1–3 pF/cm)
Connector/via parasitics	~1–5 pF
Total estimate	~30–70 pF

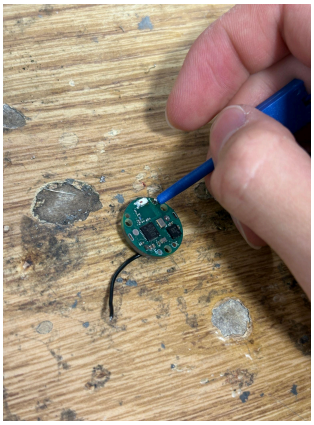
For a worst-case bus capacitance of 70pF, the maximum resistance we can use is 16.9kΩ. For an additional conservative estimate of 100pF, the maximum resistance we can use is around 11.8kΩ.

Crystal Oscillators

The EFR32 offers software programmable load capacitances on oscillators, which means that there is no need for external load capacitors. We have two external oscillators for the HFXO and LFXO: one for BLE and one for system operation, respectively. To conserve more power,

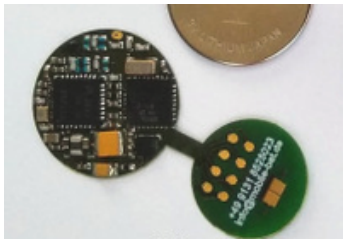
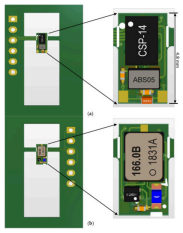
when it comes down to the nano amperage, it would be better in future designs to simply use the internal RC oscillator for the LFXO and pragmatically change to the external oscillator for BLE signal generation when necessary.

Debug



For debugging, in our initial design, we employed the use of debug pads to solder 32 wires onto. However, since we lacked 32-gauge wire in the open lab, we used a higher gauge, ripping the pads. This caused us to order testpad hooks to solder onto the pads, and tiny probe-to-headerpin connectors to thus debug the board. In the future, we would recommend using a JTAG header or a Mini Simplicity Connector pinout.

Additionally, for ultra-small board development, we recommend using no more than two layers, minimal ground planes, a wire-based antenna instead of an oscillator antenna, and a separate board that one can cut off from the main board that has pads for debug purposes. Pictured below is an image of what we describe:

a)  b) 

Source article for a) | *Source article for b)*

Hardware Layout & Routing Justifications

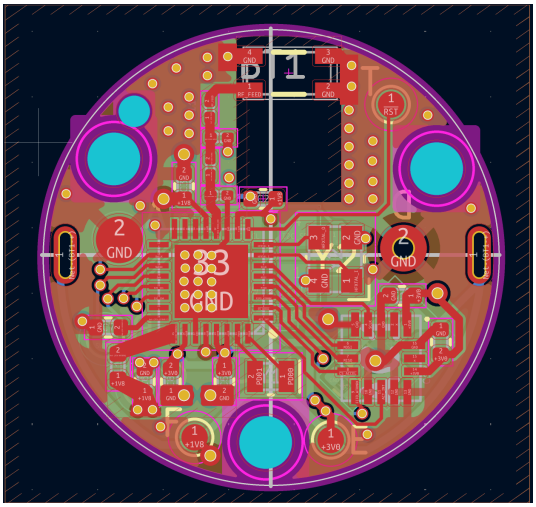


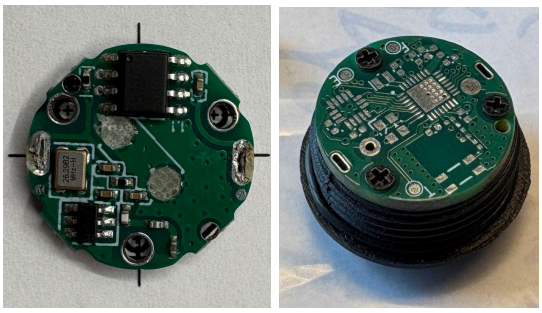
Image of PCB layout, viewed from the top, with copper pour zones filled.

Constraints

Board constraints were derived from JLCPCB's website. Impedance matching was done through their charts with a "no requirement" setup.

Dimensions

Given the fact that we were using the mechanical casing of another TPMS that we found, we had to ensure that the location of the holes and the dimensions of the board were correct for the casing. With careful measurements and 1:1 printing + drill hole verification, we managed to get a perfect fit with our first order.



Left: Image of commercial TPMS on drill hole spots retrieved from KiCAD design. Right: Our TPMS with perfect mounting on a commercial TPMS case.

Thus, in our most recent design, the board dimensions were 17.4mm, and the mounting holes were 1.5mm.

Power

Power lines are slightly thicker in case of current surges. In professional standards, one can calculate potential difference drops caused by the copper traces using Altium and decide on the trace thickness.

There is one 3V3_unprotected trace that forms a semicircle on the bottom layer of the board. RF qualities are not affected. We have considered an alternative, which is to make the entire bottom plane 3V3 unprotected to reduce the need for a trace, but that would reduce the Bluetooth signal quality due to one less ground plane and thus return path.

For decoupling capacitors, 1-2 vias were placed near the pads, ensuring a fast return path and good decoupling.

RF

Since Bluetooth is involved, there were special precautions with routing and design. We consulted the Silicon Labs EFR32xG22 layout design guide on page 31 for matching tips and conventions for good BLE signal quality. We also consulted the antenna's datasheet for clearance zone dimensions.

Since we had access to the devboard design files, we also referenced their designs to make sure that our impedance and circuit matching were done optimally.

To summarize our design choices that were derived from the layout design guide, we ensured that an adequately sized ground clearance zone was placed around the matching circuitry and below the antenna, we routed the GND pad of the first shunt capacitor in the matching network to the GND plane of the MCU, and we ensured proper return paths through via stitching of the ground planes.

Firmware Power Management Strategy

To design this system, we employed the use of energy modes, the EMU (energy management unit), and associated features that come with it. Examine section 12 of the reference manual for information.

Recommendations

We recommend that the software implementation of production systems develop their own libraries and protocol drivers from the ground up. This would take a lot of R&D, but reduces the bluff that Simplicity studio drivers add. Or, at the very least, use custom power management scripts instead of the built-in power management system by Simplicity studio generators.

We also recommend disabling unused components and peripherals on the MCU, as they consume an inordinate amount of power.

We found that the EFR32G1 uses less current during BLE transmissions, according to the spec sheet. Also, in general, FSK and OOK modulation methods seem to have less current consumption than BLE.

If assembly or raw C were to be used, we also recommend simpler MCUs to be programmed as they have shorter datasheets, fewer features that are not-so-useful in applications in low-power design, and the designer has more control over the firmware. Options include PIC 8-bit controllers or 16-bit controllers.

FSM design

Since software is currently in the midst of development, we lay out the design that we attempt to achieve with our FSM.

State 0 - Initialization

After boot or power-on reset (POR), the chip executes startup code. The following steps are designed to be performed before entering the application's main loop and FSM. However, many of these design considerations are already implemented in the preexisting libraries and packages that Silicon Labs provides, which makes the content below procedures that can be implemented when starting from scratch.

First, we want to enable the DCDC feature for 1.8V, since the DCDC bypass switch is on by default, which means VREGVDD is shorted directly to DVDD. Additionally, we set voltage scaling to VSCALE2 (for the internal circuitry, 1.1V) for EM0, but we can lower it for more power conservation.

To configure the DCDC converter directly (instead of using the given libraries and firmware already developed by Silicon Labs for the EFR32), reference 12.3.8.2 of the reference manual.

Next, we read the EMU_RSTCAUSE register. If the reset was caused by EM4 wake up, read previous context values from BURAM (persistent RAM that does not get wiped from EM4 power down) and transition directly to State 1.

Then, we initialize peripherals on the MCU - USART, I2C, SPI, TIMER, RTCC, LETIMER, IADC, WDOG.

We also configure the EM4 wakeup-capable GPIO pin for wakeup. This means setting the input glitch filters, enabling EM4 wake up on that specific pin, what type of interrupt and polling is wanted (rising edge), enabling interrupt generation, and clearing previous interrupt registers.

If the reset was caused by POR, we initialize the ADXL362 over SPI using a soft reset (writing 0x52 to register 0x1F), followed by device ID verification. The sensor is then configured to operate in wake-up mode, making it a motion switch that runs at 6Hz and draws 270nA. Activity and inactivity thresholds can be set, in addition to the activity and inactivity duration thresholds. We also enable loop mode, which means we do not have to service any interrupts, conserving even more power. We deliberately do not want any measurements, so we do not enable autosleep, which automatically transitions from deep sleep to measurement mode. Finally, we remap an interrupt pin to be for the AWAKE bit (ACTIVITY = 1, INACTIVITY = 0), and then start the sensor. For power conservation purposes, if an IMU is used in other design products, we recommend using the ADXL362's FIFO buffer to reduce polling intervals, thus reducing the power consumption.

Additionally, we initialize the MS5837 through I2C. This is slightly easier, as the only read that takes place during initialization is loading prom data into memory so it can be used to convert sensor polls into corrected data later. Additionally, the OSR is configured during reads; however, it can also be configured during startup, which reduces the amount of I2C writes.

Finally, we transition to State 1.

State 1 - Activity

During this state, the car is moving, and the chip cycles between EM0 and EM2. The EFR32 might have just woken up from an EM4 reset and was woken up through a hardware interrupt by the ADXL362 after detecting motion, or by power-on reset.

During this state, the chip first polls the AWAKE bit to determine whether inactivity is detected; if so, we enter State 2, if not, it polls the MS5837 to transmit the data in EM0. Then, in between measurements, the chip goes to soft sleep in EM2 with an RTCC timer that will wake up the chip to poll for pressure and temperature data again. This happens at a predetermined duty cycle that is up to the engineering team.

State 2 - Transition to EM4

The ADXL362's AWAKE bit goes low whenever inactivity is detected. Once this happens, we initialize the EM4 entry workflow.

First, we save state variables to BURAM (backup ram). Second, we verify there are no subprocesses that are running. Then, we verify that the EM4 wakeup pin is configured correctly, clear GPIO interrupt flags, and disable the DCDC converter.

Finally, we enter EM4 by writing the magic sequence given in the reference manual to EMU_EM4CTRL.EM4ENTRY. System "transitions" to State 3

State 3 - Inactivity

During this state, all systems are in deep sleep or powered down. Total system current consumption is a few microamps.

FSM Power Expectations

All estimates assume a 3.0V coin cell, DC-DC enabled (1.8V), VSCALE0 in sleep, VSCALE1/2 in active. We will use tables to represent an estimated duration percentage for which each component will run for a period of operation.

Summary

State	Current
0 (Init)	~0mA
1 (Activity)	42.34µA

State	Current
2 (Transition)	~0mA
3 (Inactivity)	420nA

Assuming that activity takes up 5% of the operational life cycle in a TPMS sensor, with 95% being inactive, the TPMS consumes a weighted average of 2.516µA.

A CR1632 has 120mAh of charge. This means that the system can theoretically run for 5.4 years. If we are continuously using the TPMS sensor, then it will last us 283 days.

State 0 - Init

The system would take 8ms to go from POR to code execution, or 1.83ms from EM4 to code execution. At a maximum, this would consume 5mA of power, for less than 2ms in typical duty cycles.

State 1 - Activity

Let us assume that one cycle takes ~1000ms to complete. A cycle is defined by the action of the MCU waking up from EM2 to polling the sensor in EM0 to going back to sleep to EM2.

Component	Current	Active Duration
EFR32 EM0 (38 MHz, processing)	~1 mA (26 µA/MHz × 38 MHz)[27]	~3ms
EFR32 BLE TX (0 dBm)	3.4mA[28]	~9.9ms[29]
MS5837 conversion (256 OSR, N=3)	1.25mA * 3 = 3.75mA	1.75ms[30]
ADXL362 wakeup mode (6 Hz)	270nA[31]	1000ms
EFR32 EM2 (PD0B active, LFXO)	1.94µA[32]	~985ms

[27] EFR32 Datasheet Table 4.6.1
[28] EFR32G22 Beacon TX Current Profile Plots & EFR32 Datasheet Table 4.6.4
[29] EFR32G22 Beacon TX Current Profile Plots Table 2.4 | Repeated 3 times, once (3.344ms) for one of three channels.
[30] For 3 samples, MS8537 Datasheet ADC Performance Table, Adding ~150µs of I2C transaction time.
[31] ADXL362 Datasheet (This number is everywhere)
[32] EFR32 Datasheet Table 4.6.2

Thus, the estimated current consumption for State 1 is: 1 mA * 0.003 + 3.4mA * 0.009 + 3.75mA * 0.00175 + 0.000270mA * 1 + 0.00194 * 0.985 = 0.04234 mA = 42.34µA.

State 2 - Transition

State two can be treated as a continuous run in EM0 for 10 microseconds[33]. This makes it negligible in overall calculations.

State 3 - Inactivity

Component	Current
EFR32 EM4 (No BURTC, no LFXO)	0.14µA[34]
ADXL362 wakeup mode (6 Hz)	270nA
MS5837 standby	0.01µA
IRLML6401 (PMOS)	<0.1nA

Thus, the estimated current draw of the system in EM4 is 420nA.

Bluetooth Specifics

Payload Design

This section details the 10-byte payload that is intended to be sent out once on three different bluetooth channels. We take inspiration from the reverse engineered TPMS system's protocols and encoding formats.

Typically a BLE 4.x advertisement payload can carry 31 bytes, but we use 10 bytes since it is sufficient for our use case. We can even optimize the design by sending bytes 3,4,5,7, and 9.

Byte	Meaning
0	ISC ID low byte
1	ISC ID high byte

[33] EFR32 Datasheet Table 4.10
[34] EFR32 Datasheet Table 4.6.2Bluetoothreverse-engineered,

Byte	Meaning
2	Protocol version
3	Node address (0x00=FL, 0x01=FR, 0x10=RL, 0x11=RR - commercial)
4	Pressure byte (raw_kpa_gauge / 3.44 - commercial)
5	Temperature byte (temp_c + 50 - commercial)
6	Warning flags
7	Battery byte (ADC_mV / 20 - commercial)
8	Sequence counter
9	CRC8 (available on EFR32 hardware, yay!)

For byte 4, the commercial raw_byte * 3.44 scheme maps a single unsigned, received byte to 0-877kPa. Typical car tire pressures are 200-350 kPa. This is 3.44 kPa/LSB resolution. This is more than adequate for tire pressure monitoring, where the industry standard is ±7kPa.

For byte 5, byte - 50 maps 0-255 to -50°C to +205°C. Tire operating range is -40°C to +125°C, which fits perfectly with the scheme's range.

Bit	Warning Flag Meaning
0	LOW_BATTERY
1	SENSOR_FAULT
2	RESERVED
3	AIR_LEAK
4	PRESSURE_HIGH
5	PRESSURE_LOW
6	TEMPERATURE_HIGH

Bit	Warning Flag Meaning
7	RESERVED

Expected Power Profiles during TX of EM0

The EFR32G22 has a good guide and power profile PDF by Silicon Labs that provides example power profiles for BLE beacon (advertisement only, non-connectable) transmissions. These are equivalent to what our system will do.

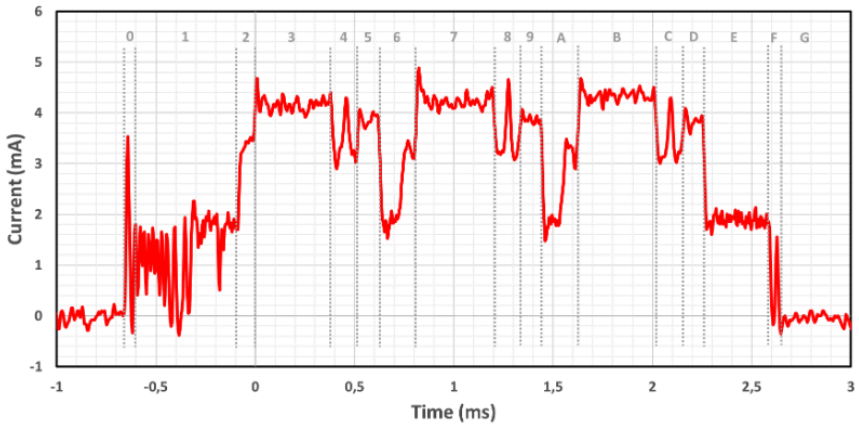


Figure 2.6. Current Profile for Series 2 EFR32BG22 SoC Running SoC-iBeacon Example Application (Active Only)

Expected relative magnitude of the BLE current consumption graph during active TX transmission.

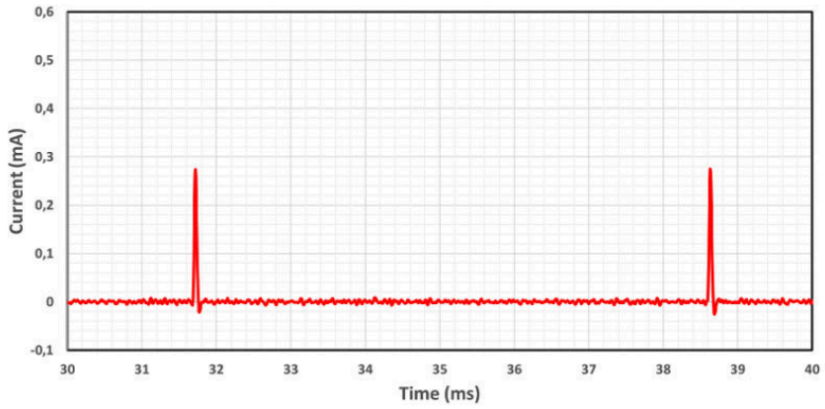


Figure 2.9. Current Profile for Series 2 EFR32BG22 SoC Running SoC-iBeacon Example Application (Sleep)

Expected relative magnitude of the BLE current consumption graph during the EM0 cycle. Each spike is during TX.

Firmware Reverse Engineering Aftermarket TPMS

In order to better understand commercial TPMS systems, Eddie reverse-engineered one of them that he bought from Amazon. This system came with a USB-based receiver contraption, making reverse engineering slightly easier, as there was a likelihood that it could be reverse-engineered in the first place (turns out it did).

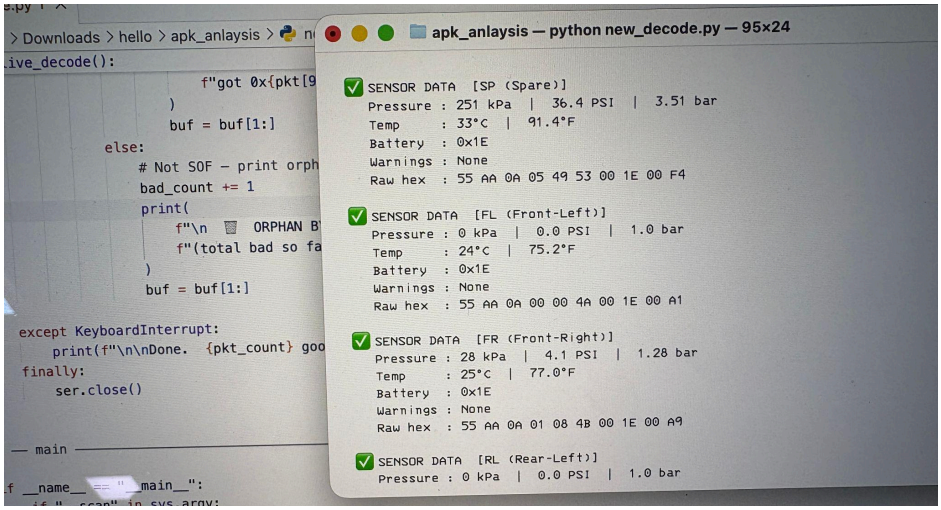
A timeline of events follows: Eddie notices that a USB drive came with the TPMS box that contained the system. Within it, there exist APK files. He suspects that this USB-based receiver is supposed to be connected to an Android-based system. Amazon confirms his suspicions.

On the receiver side, Eddie plugs it into his laptop. The USB device is named “USB Serial”. This makes Eddie suspect that there is serial data. Turns out there is. Eddie tries decoding the USB serial data by hard-bashing the protocol and port numbers before analyzing anything in the APK files.

Eddie has no success, so he then decompiles the APK file, searches for relevant files (with success), uses Claude to analyze them, and slowly reconstructs the protocol based on what was found. Turns out there was a handshake protocol and multiple levels of decoding schemes involved.

Overall, this is a dramatic oversimplification of what actually happened, but it provides an adequate overview.

Feel free to review the raw files that Eddie decoded/analyzed. The final working script is in the folder titled “apk_anlaysis” (yes, there is unfortunately a spelling mistake). Link to the folder w/ all the files.

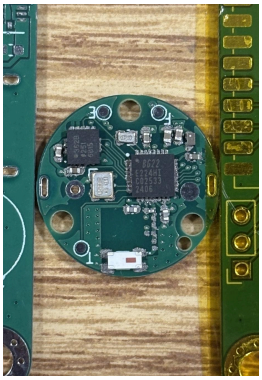
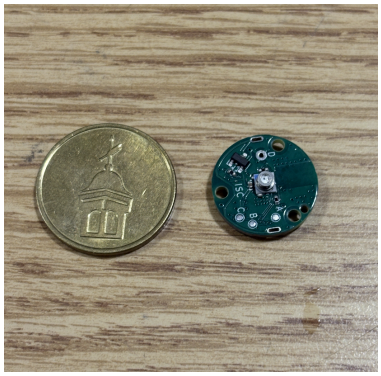


Successful decoding of data from the FR sensor that was connected to a test tire in the OpenLab

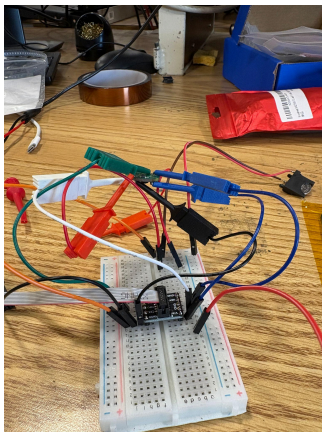
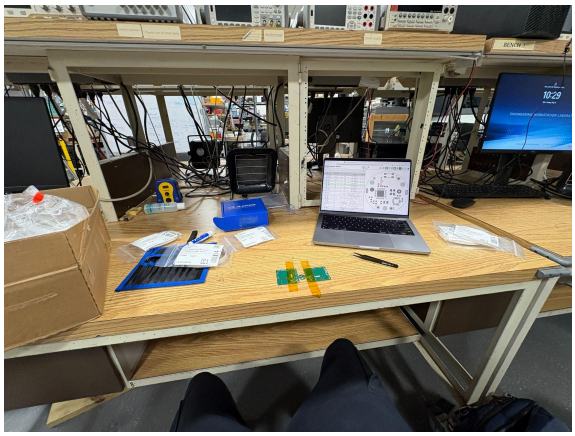
Hardware Bring Up

Below details the hardware bring-up process undertaken near the end of the semester, primarily consisting of board assembly, initial power validation, and early-stage SWD debugging attempts.

Upon receiving the first batch of boards, Eddie assembled a complete prototype in approximately three hours, which is a notable success, particularly given the precise tolerances achieved with the casing, screws, and overall form factor. However, the debug pads, which were intended to accept soldered 22 AWG wire for SWD access, proved too fragile and ripped easily, effectively resetting all progress on that board. A secondary issue was discovered in the BOM: 100pF 0402 capacitors had been mistakenly ordered instead of the required 100nF caps, though replacements were sourced from the open lab. The pressure sensor membrane was also accidentally damaged during assembly, rendering that unit non-functional, though the MCU, accelerometer, crystal oscillators, and antenna were salvaged.

A second assembly session followed, this time using 30 AWG wire and grabber mounts, which proved far more practical and modular for attaching to the small debug pads. During this session, the DCDC converter output was reading only a few millivolts and continuously decaying to zero rather than the expected 1.8V bypass behavior following POR. Swapping to a fresh MCU resolved the power issue, but SWD communication remained elusive.

With a functional power rail established, Eddie attempted to connect a J-Link debugger to the board's SWD pads (SWDCLK, SWDIO, RST, VREF, GND) and query the device through Simplicity Commander. The chip returned no acknowledgment. Probing revealed that SWDCLK was only reaching approximately 1.5V despite VREF being tied to the 3.0V rail, which is a voltage level inconsistent with the logic thresholds of the EFR32BG22. The issue appeared intermittently self-correcting before returning, and wire length was found to have a measurable impact on signal integrity at clock speeds above 1000 kHz, likely due to parasitic inductance. Grounding anomalies were also observed, with the oscilloscope showing the SWDCLK line swinging into negative voltages. Reducing the SWD clock speed down to 4 kHz via Simplicity

Commander's terminal brought the SWDCLK voltage up to approximately 2.1V, which was within the detection threshold for 3.0V logic, but the device still did not respond. At the time of writing, the leading hypothesis is a ground plane integrity issue stemming from the board's compact size, which may be causing noise and reference instability on the SWD lines.

Recommendations and Next Steps

The most immediate priority for the next cycle is a PCB redesign incorporating a larger ground plane and much more relaxed board dimensions (2’’x2’’), which should directly address the suspected SWD signal integrity and ground reference issues encountered during bring-up. It would also be worth upgrading the debug pads to ENIG (gold-plated) finish on the old PCB (if they are manufactured again) while completely ditching the debug pads in favor of debug hooks and pin outs on the 2’’x2’’ big prototype board. Furthermore, it would be wise to include an actual JTAG programming header or connector holes that we can solder pins onto for programming.

A board re-order timed just before the fall semester begins would allow for a clean bring-up session at the start of the new term.

On the firmware side, the team has already demonstrated successful I2C, SPI, and BLE communication with breakout board sensors using Simplicity Studio drivers with custom sensor interfaces for the MS5837 pressure sensor and ADXL362 accelerometer. The next logical step is to do fine-grained power control, Bluetooth testing, and EM2/EM4 compatibility testing on the EFR32BG22. Alongside this, the team should begin designing and implementing the core finite state machine (FSM) that governs the sensor's operating modes, wake conditions, and BLE advertisement intervals.

For hardware power architecture guidance, the team & AbbVie should look closely at animal tracker tag research. Eddie has been doing research during the past summer regarding tag design and engineering, and many low-power principles from there apply similarly to the design and operation of low-power systems in commercial systems. A few research papers Eddie recommends examining: link, link, and link.